

- 🎮 Blog de Consolas y Videojuegos (Proyecto React - Requisitos Cumplidos)
 - 📋 Resumen de Componentes
 - ✅ Cumplimiento de Requisitos Técnicos Obligatorios
 - A. Componentes (Mínimo 8)
 - B. Estados (useState)
 - C. Reutilización obligatoria

Blog de Consolas y Videojuegos (Proyecto React - Requisitos Cumplidos)

Este proyecto es una aplicación web de blog construida con React, enfocada en la componentización, el flujo de datos unidireccional (props) y la gestión de estado con `useState`, cumpliendo con todos los requisitos técnicos obligatorios del enunciado.



Resumen de Componentes

La aplicación utiliza un total de **9 componentes funcionales** (más del mínimo de 8 requerido), garantizando la separación de responsabilidades y la reutilización:

- `App` (Raíz)
 - `Header` (Sin Props)
 - `Footer` (Sin Props)
 - `Sidebar`
 - `ConsolaList` (Lista)
 - `ConsolaCard` (Reutilizable/Estado Local)
 - `LatestConsolesCard` (Reutilizable/Personalización)
 - `GameCard` (Reutilizable)
 - `AddCommentsForm` (Formulario Controlado)
 - `ToggleCommentsButton` (Callback)
 - `CommentsList` (Callback)
-



Cumplimiento de Requisitos Técnicos Obligatorios

A continuación, se detalla cómo se cumple cada requisito del enunciado, especificando el componente, el fragmento de código relevante y la lógica empleada.

A. Componentes (Mínimo 8)

Requisito	Componente(s)	Código/Lógica Implementada
A. Componente Raíz	<code>App.tsx</code>	<code>App</code> gestiona el estado global <code>filterTag</code> y pasa las funciones y datos necesarios a <code>Sidebar</code> y <code>ConsolaList</code> .
B. Al menos 1 componente sin props	<code>Header.tsx</code> , <code>Footer.tsx</code>	Ambos componentes son estáticos y no reciben ninguna <code>prop</code> .
C. Al menos 2 componentes reutilizables	<code>ConsolaCard.tsx</code> , <code>GameCard.tsx</code> , <code>LatestConsolesCard.tsx</code>	<code>ConsolaCard</code> se reutiliza en <code>ConsolaList</code> mediante un <code>.map()</code> . <code>GameCard</code> se reutiliza dentro de cada <code>ConsolaCard</code> para listar los juegos.
D. Al menos 2 componentes que reciban props para personalizar su apariencia/contenido	<code>ConsolaCard.tsx</code> , <code>LatestConsolesCard.tsx</code>	Ambos utilizan la propiedad <code>consola.color</code> para establecer un borde personalizado, demostrando personalización basada en <code>props</code> .

Requisito	Componente(s)	Código/Lógica Implementada
E. Al menos 1 componente formulario controlado	<code>AddCommentsForm.tsx</code>	Gestiona el valor del <code>textarea</code> internamente con <code>useState</code> .
F. Al menos 1 componente que reciba una función para cambiar un state por props (callback)	<code>Sidebar.tsx</code> , <code>AddCommentsForm.tsx</code> , <code>ToggleCommentsButton.tsx</code>	Caso clave (Global): <code>Sidebar</code> recibe <code>onFilter={({tag}) => onFilter(tag)}</code> para modificar el estado <code>filterTag</code> definido en <code>App</code> .
G. Componente que muestre una lista de elementos (usando <code>.map()</code>)	<code>ConsolaList.tsx</code>	Renderiza múltiples instancias de <code><ConsolaCard /></code> utilizando el método <code>.map()</code> sobre el array de <code>consolas</code> .
H. Componente que actúe como "panel de información" o "visor" de un elemento seleccionado.	Requiere mejora (Ver Nota )	Actualmente, el detalle se muestra dentro de cada <code>ConsolaCard</code> . Para cumplir estrictamente con un "visor de elemento seleccionado" (estado global), se debería: 1. Añadir <code>selectedConsola</code> a <code>App</code> y 2. Crear un componente <code>ConsolaViewer</code> que se muestre condicionalmente en <code>App</code> si <code>selectedConsola</code> no es <code>null</code> .

B. Estados (useState)

Requisito	Componente(s)	Código/Lógica Implementada
1. Estado local obligatorio en varios componentes (Mínimo 2)	ConsolaCard.tsx, AddCommentsForm.tsx	ConsolaCard: const [comments, setComments] = useState(...), const [showComments, setShowComments] = useState(...). AddCommentsForm: const [text, setText] = useState("").
2. Estado compartido entre varios componentes (Mínimo 1 caso)	App.tsx	const [filterTag, setFilterTag] = useState(""); Este estado afecta a qué consolas se muestran en ConsolaList.
3. Al menos 1 componente debe leer y otro modificar el estado compartido	Sidebar.tsx (Modifica) y ConsolaList.tsx (Lee)	Modifica: Sidebar recibe onFilter (que es setFilterTag de App) y lo usa en sus botones de filtro. Lee: ConsolaList recibe el array consolas ya filtrado por el estado filterTag de App.

C. Reutilización obligatoria

Requisito	Componente(s)	Código/Lógica Implementada
Demostrar Reutilización	ConsolaCard.tsx, GameCard.tsx, LatestConsolesCard.tsx	La reutilización se demuestra claramente a través del uso de .map() en los componentes ConsolaList y ConsolaCard, y en Sidebar.